

Day 6 — 边缘智能与整机联调 (F407VET6 新核心板版)

已按新核心板更新：I2C 总线恢复小节改为软 I2C（工程 bsp_i2c_soft.c / BSP_I2C_Soft_Recover）；栈溢出指示 LED 改为板载 PA6；其余振动特征/诊断/联调逻辑不变

今日目标：整机连续运行 30 分钟无崩溃、无数据异常、OLED 显示正常、MQTT 数据持续上传、告警功能可正常触发和恢复。振动特征提取正常，故障诊断三态（正常/注意/异常）判断准确。

开发顺序：振动特征提取 → 故障诊断规则 → 诊断显示 → 整机联调 → 栈溢出检测 → I2C 恢复 → 断网补传 → Cube.AI预留 → 接线整理。

第一步：振动特征提取

QMI8658 输出瞬时加速度，需要在一段时间窗口内计算统计特征（RMS/峰值/峭度/波形因子/峰值因子）。工程中对应 usrcode/APP/Src/app_vibration.c（app_vibration.h）。

```
/* 关键定义（对齐工程 app_vibration.h） */
#define VIB_WINDOW_SIZE 100      /* 100个采样点，10ms周期=1秒窗口 */
typedef struct {
    float rms;          /* 均方根，反映振动能量 */
    float peak;         /* 峰值，最大偏离 */
    float kurtosis;     /* 峭度，反映冲击特性(正常≈3，故障>3) */
    float crest_factor; /* 峰值因子 = 峰值/RMS，早期故障敏感 */
    float waveform_factor;
    float mean_abs;
} VibrationFeatures_t;
```

特征物理意义：(1) RMS 反映振动总能量，轴承磨损/不平衡导致 RMS 升高；(2) 峭度反映冲击成分，正常≈3，点蚀/剥落时飙到 5-10；(3) 峰值因子对早期故障敏感；(4) 这些是旋转机械故障诊断的经典指标，答辩可讲清每个特征的物理意义。

第二步：简易故障诊断规则（三态：正常/注意/异常）

工程对应 usrcode/APP/Src/app_fault.c（app_fault.h），规则引擎基于 振动特征+温度+电流：

```
#define FAULT_NORMAL    0
#define FAULT_WARNING   1
#define FAULT_ABNORMAL  2
/* 故障类型位图 */
#define FAULT_TYPE_OVERHEAT (1 << 0) /* 过热 */
#define FAULT_TYPE_OVERLOAD (1 << 1) /* 过载 */
#define FAULT_TYPE_VIBRATION (1 << 2) /* 振动异常 */
```

```
#define FAULT_TYPE_IMBALANCE  (1 << 3)  /* 不平衡（RMS高+峭度正常） */
#define FAULT_TYPE_BEARING    (1 << 4)  /* 轴承故障（峭度高） */

/* 规则要点：
 * 1) 温度>TEMP_ABNORMAL → ABNORMAL(过热)； >TEMP_WARNING → WARNING
 * 2) 电流>CURR_ABNORMAL → ABNORMAL(过载)； >CURR_WARNING → WARNING
 * 3) 峭度>6 或 峰值因子>6 → ABNORMAL(轴承故障)； >4 → WARNING(早期磨损)
 * 4) RMS高+峭度正常(≈3) → ABNORMAL(不平衡/不对中)
 * 5) 多个注意级故障叠加 → 升级为异常
 * 6) 置信度：ABNORMAL=0.9 / WARNING=0.7 / NORMAL=0.95 */
```

第三步：诊断结果 OLED 显示与 MQTT 上报

```
/* DisplayTask: 实时数据页底部显示诊断状态（工程 app_oled / app_fault） */
switch (diag.status) {
    case FAULT_NORMAL: OLED_ShowString(0, 7, "Status: Normal");    break;
    case FAULT_WARNING: OLED_ShowString(0, 7, "Status: Warning!"); break;
    case FAULT_ABNORMAL:OLED_ShowString(0, 7, "Status: ABNORMAL!!");break;
}
/* MQTT JSON 增加诊断字段 */
"alarm": %d, "status": %d, "fault_mask": %d, "diag": "%s"
```

第四步：整机联调检查清单

- 1. 上电后所有外设初始化成功（串口打印各模块 Init OK）
- 2. FreeRTOS 5个任务全部创建成功，调度器正常运行
- 3. OLED 实时显示三路传感器数据，数值合理
- 4. 按键 A 切换三个页面正常，按键 B 控制继电器
- 5. 手捂 SHT30 升温 → 告警触发 → 蜂鸣器/继电器/OLED/MQTT 联动
- 6. 松手降温 → 告警自动恢复（10% 迟滞）
- 7. MQTT 持续收到 JSON 数据，每 2 秒一条
- 8. 断开 WiFi 30 秒 → 自动重连 → 数据恢复上传
- 9. PyQt5 上位机显示实时数据和曲线
- 10. 上位机点击继电器按钮 → 单片机继电器动作
- 11. 振动特征提取正常，故障诊断三态判断合理
- 12. 连续运行 30 分钟无死机、无数据异常

问题	排查方法	解决方案
任务越多越容易死机	开启栈溢出检测，看哪个任务溢出	增大对应任务栈大小
I2C 设备偶尔读不到	检查总线是否有死锁	调 BSP_I2C_Soft_Recover() 总线恢复（软I2C）
MQTT 发布时系统卡顿	MqttTask 优先级太低或 AT 指令阻塞	增大 MqttTask 栈，AT 指令加超时
告警频繁触发/解除	数值在阈值附近抖动	增大迟滞区间或加强滤波
OLED 显示偶尔花屏	总线冲突或刷新被打断	OLED 操作加互斥锁

串口打印乱码	波特率不匹配或配置错误	确认 USART1 波特率 115200
--------	-------------	----------------------

第五步：栈溢出检测与内存优化

```

/* FreeRTOSConfig.h */
#define configCHECK_FOR_STACK_OVERFLOW 2    /* 方法2：更严格的检测 */
#define configUSE_MALLOC_FAILED_HOOK 1

/* 栈溢出钩子（freertos.c）—— 指示 LED 用板载 PA6（LED2，低电平点亮），勿用 PA5（蜂鸣器） */
void vApplicationStackOverflowHook(TaskHandle_t xTask, char *pcTaskName)
{
    printf("STACK OVERFLOW: %s\r\n", pcTaskName);
    while (1) { HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_6); HAL_Delay(100); }
}
void vApplicationMallocFailedHook(void)
{
    printf("MALLOC FAILED\r\n");
    while (1) { HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_6); HAL_Delay(200); }
}

```

栈优化建议：(1) usStackHighWaterMark < 50 words 有溢出风险，增大栈；(2) 避免任务内大局部数组，改全局/静态；(3) printf 和 %f 浮点耗栈大，MqttTask 大量 snprintf 给 1024W；(4) configTOTAL_HEAP_SIZE 建议 20KB 以上（F407 有 192KB RAM）。

第六步：I2C 总线恢复机制（软 I2C 版，新核心板）

新板差异：本工程 I2C 全部为软件模拟（bsp_i2c_soft.c，总线1=PB8/PB9 挂三传感器；总线2=PD6/PD7 挂 OLED），无硬件 I2C 外设，总线恢复直接在 GPIO 上打 9 个时钟脉冲即可，工程已封装：`void BSP_I2C_Soft_Recover(void)`。

```

/* bsp_i2c_soft.h 提供的总线恢复 API */
/**
 * @brief I2C总线恢复 - 手动产生9个SCL时钟脉冲 + 停止位，释放被拉低的SDA
 * @note 由应用层在"连续读取失败N次"时触发
 */
void BSP_I2C_Soft_Recover(void);

/* 应用层使用（app_sht30 / app_qmi8658 / app_ina226 的失败计数逻辑） */
static uint8_t i2c_fail_count = 0;
if (BSP_SHT30_Read(&sht) != 0) {
    i2c_fail_count++;
    if (i2c_fail_count >= 5) {
        printf("I2C failed %d times, recovering...\r\n", i2c_fail_count);
        BSP_I2C_Soft_Recover();    /* 打9个SCL脉冲 + STOP，释放SDA */
        BSP_I2C_Soft_Init();      /* 重新初始化引脚为推挽/输入模式 */
        i2c_fail_count = 0;
    }
}

```

```
} else {
    i2c_fail_count = 0;
}
```

原理说明：某从机异常拉低 SDA 不释放 → 主机在 SCL 上打 9 个时钟脉冲，让从机完成当前字节时序后释放 SDA，再发 STOP 恢复空闲。总线1 三传感器共用，任何一个卡死都会影响全部，务必保留此机制。

第七步：断网补传（选做）

```
/* app_offline.c: RAM 环形缓冲区（300条=10分钟） */
if (MQTT 发布失败) { OFFLINE_Push(&offline_buf, &data); connected = 0; }
if (connected && OFFLINE_Count(&offline_buf) > 0) {
    while (OFFLINE_Pop(&offline_buf, &old_data) && count < 50) {
        MQTT_Pub_SensorData(&old_data); count++;
        osDelay(50); /* 避免发布太快 */
    }
    printf("Offline data backfilled: %u\r\n", count);
}
```

第八步：Cube.AI 接口预留（选做）

```
/* app_fault.h 预留 AI 接口（USE_CUBE_AI 宏） */
#ifdef USE_CUBE_AI
void    FAULT_AI_Init(void);                                /* ai_motor_fault_create */
uint8_t FAULT_AI_Infer(const float *input_features, float *output_prob); /* ai_motor_fault_run */
#endif
/* 答辩说明：已预留 Cube.AI 接口，可后续采集真实故障数据训练一维 CNN/LSTM 模型部署 */
```

第九步：接线整理与供电检查

- 拍照记录：各角度拍清接线照片，防碰掉后接不回
- 杜邦线理线：同组捆一起（I2C一组/电源一组/GPIO一组），留余量，标注关键线
- 供电检查：ESP8266 独立 3.3V（≥300mA，AMS1117 单独一路）；继电器模块 5V；所有 GND 共地
- 防碰措施：核心板铜柱固定；杜邦线接头热熔胶固定；准备备用 USB 线和 5V 适配器

现象	排查方向
整机运行后 HardFault	(1)栈溢出 (2)数组越界 (3)空指针 (4)中断优先级冲突 (5)看 LR/PC 定位
I2C 总线死锁，传感器读不到	(1)调 BSP_I2C_Soft_Recover() (2)查是否设备拉低SDA (3)上拉 4.7kΩ (4)降速
振动特征异常	(1)窗口是否填满 (2)输入是否为加速度模值(含1g重力) (3)峭度除零保护 (4)采样率稳定

诊断状态跳变	(1)加滤波 (2)阈值合理 (3)连续N次异常才升级 (4)迟滞足够
MQTT 发布阻塞卡顿	(1)AT 超时 (2)MqttTask 优先级 (3)2秒一次足够 (4)DMA 接收正常
断网后不补传	(1)离线缓冲正确存入 (2)重连后调补传 (3)不被新数据阻塞 (4)缓冲区大小
OLED 花屏偏移	(1)总线冲突加互斥锁 (2)初始化参数 0xDA/0x12 (3)刷新被打断 (4)供电稳定
继电器误触发	(1)告警逻辑 (2)远程命令误解析 (3)光耦隔离 (4)GPIO 默认电平=断开

Day 6 完成标志：整机连续运行 30 分钟无崩溃；OLED 三页面切换正常；振动特征每秒更新数值合理；故障诊断三态准确（正常显示 Normal，手捂升温显示 Warning/Abnormal）；MQTT 持续上传含诊断信息；断网 1 分钟恢复自动重连并补传；接线整理完毕拍照记录；演示流程跑通：正常运行 → 模拟异常 → 告警联动 → 远程查看 → 恢复。